Introducing Mtac2: first-class tactics with first-class types in Coq

Beta Ziliani
CONICET / FAMAF - UNC

Joint work with **Jan-Oliver Kaiser, Robbert Krebbers, Yann Régis-Gianas, Derek Dreyer**, with contribs from Béatrice Carré, Jacques-Pascal Deplaix, Thomas Refis, and others.

A proof in Coq

```
Theorem tl_length_pred : forall l: list nat,
  (length l) - 1 = length (tail l).
Proof.
  destruct 0; [ idtac | intros n l ] ; reflexivity.
Qed.
```

The **Proof Script**:
here we write
tactics and **tacticals**

Tactics in Coq

#1: The Old Times

- A step in the elimination / introduction rules of the calculus:

intro x

$$\frac{x{:}P \vdash Q}{\vdash P \to Q}$$

- A *program* decomposing a goal into smaller *subgoals*:

apply *lemma*

$$\frac{\vdash P \qquad \vdash Q}{\vdash R}$$

lemma : P -> Q -> R

All written in OCaml

- A *program* to solve problems of a specific domain:

omega

$$\frac{}{x > 0 \to x + y > 0}$$

Tactics in Coq

#1: The Old Times

- **Tacticals:** A *composition* of primitive tactics.

```
eapply (tac_wp_pure _ _ _ _ (fill K e'));
 [ apply _                                  (* PureExec *)
 | try fast_done                            (* The pure condition for PureExec *)
 | apply _                                  (* IntoLaters *)
 | wp_expr_simpl_subst;                     (* new goal *)
   try wp_value_head ]
```

Written in the proof script

(Snippet from the Iris project)

Tactics in Coq

#2 Ltac: A "New" View

- A *functional* ***meta-program*** manipulating *terms* and *goals*:

```
Ltac of_expr e := lazymatch e with
  | hl.Var ?x => constr:(Var x)
  | hl.App ?e1 ?e2 =>
      let e1 := of_expr e1 in let e2 := of_expr e2 in constr:(App e1 e2)
  | _ => match goal with
           | H : Closed [] e |- _ => constr:(@ClosedExpr e H)
           end
  end.
```

Shallowly embedded terms: **no quoting!**

Inspection of goal state

(Snippet from the Iris project)

Problems with Ltac

Ltac and Ltac2 still miss
precise types in Gallina!

- It's not a proper language:
  - It misses datatypes (e.g., no list for tactics),
  - Have no real typing (e.g., gets confused about **constr** and **ltac** in places it shouldn't),
  - What is not provided can't be coded (e.g., very limited support for goal reordering),
  - No proper error handling (e.g. just fail).
- Complex implementation:

"... the implementation of [Ltac] involves considerable changes in the interpretation of proof scripts, essentially due to the matching operators."

- Ltac2 improves the first point.

Why typed tactics? (tacticals)

```
eapply (tac_wp_pure _ _ _ _ (fill K e')); (* PureExec *)
  [ apply _                                (* The pure condition for PureExec *)
  | try fast_done
  | apply _                                (* IntoLaters *)
  | wp_expr_simpl_subst;                   (* new goal *)
    try wp_value_head ]
```

Are these the right number of subgoals?

Are we shaping the new goal as expected?

(Snippet from the Iris project)

Why typed tactics? (meta-program)

Are we missing a case?

```
Ltac of_expr e := lazymatch e with
  | hl.Var ?x => constr:(Var x)
  | hl.App ?e1 ?e2 =>
      let e1 := of_expr e1 in let e2 := of_expr e2 in constr:(App e1 e2)
  | _ => match goal with
          | H : Closed [] e |- _ => constr:(@ClosedExpr e H)
          end
  end.
```

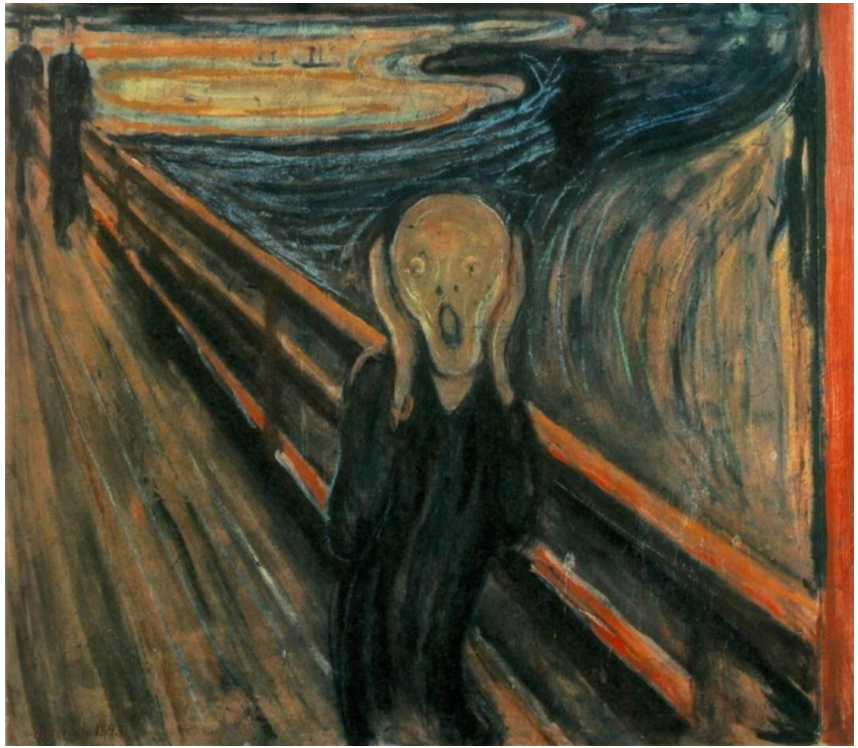
Are these the right arguments and constructors?

Are we returning the right thing?

(Snippet from the Iris project)

A typo… a late-night change…

Set Ltac Debug.



Hypothesis

Types can help us obtain robust, maintainable tactics!

Typed tactics in Mtac2 (tacticals)

```
`Δ' e2 φ <- M.evar _;
TT.apply (tac_wp_pure _ Δ' _ _ (fill K e') e2 φ _)
  <**> TT.by' T.apply_                    (* PureExec *)
  <**> TT.use (T.try fast_done)           (* The pure condition for PureExec *)
  <**> TT.by' T.apply_                    (* IntoLaters *)
  <**> (`e' <- M.evar _;                  (* new goal *)
        wp_expr_simpl_subst e'
        <**> TT.try wp_value_head)
```

THESE ARE THE RIGHT NUMBER OF SUBGOALS

WE ARE SHAPING THE GOAL AS EXPECTED

Morally, `<**> : (A -> B * goals) -> (A * goals) -> (B * goals)`

Typed tactics in Mtac2 (meta-program)

WE ARE RETURNING THE RIGHT THING

```
Definition of_expr e  : heap_lang.expr → gtactic expr := mfix1 go e :=
  mtry
    match e with
    | hl.Var x =>  T.ret (Var x)
    | hl.App e1 e2 =>
        e1 <- go e1; e2 <- go e2; T.ret (App e1 e2)
    end
  with StuckTerm =>
     H <- T.select (Closed [] e); T.ret (@ClosedExpr e H)
  end.
```

THESE ARE THE RIGHT ARGUMENTS AND CONSTRUCTORS

WE CAN'T MISS A CASE

The base: Mtac

A language for *typed* meta-programming

Typed meta-programs in Mtac

```
Definition of_expr e  : hl.expr → M expr := mfix1 go e :=
  mtry
    match e with
    | hl.Var x =>  ret (Var x)
    | hl.App e1 e2 =>
        e1 <- go e1; e2 <- go e2; ret (App e1 e2)
    end
  with StuckTerm =>
     raise (WrongTerm e)
  end.
```

Meta-effects in the monad **M**

No goal Inspection

How we do meta-programming in Mtac

- Describe the "effects" in an inductive type **M**:

```
Inductive M : Type → Prop :=
| ret : A → M A
| bind : M A → (A → M B) → M B
| mtry : M A → (Exception → M A) → M A
| raise : Exception → M A
| mfix1 : ((∀x : A. M (B x)) → (∀x : A. M (B x)))→ ∀x : A. M (B x)
| ...
```

- Execute them in an interpreter.
  - It inherits β, δ, ι, ζ reductions from Coq.

The win of Mtac

- The typechecker catches errors at an early stage.
- A full-fledged functional language, with Coq's own stdlib, notation mechanism, etc.
- Undoubtedly better than Ltac's meta-programs [1].

[1] https://gmalecha.github.io/reflections/2016/04/18/experimenting-with-mtac/

Mtac2

Redesign of Mtac with support for **tactic** development

Mtac2: Mtac + support for tactics

Mtac +

1) A new proof environment MProof.
2) New language constructs: hypotheses, constrs, abs_let, ...
3) A *first-class* representation for goals within Coq.
4) (At the moment) two tactic types to describe two levels of correctness.
5) (Some) integration from-and-to Ltac.

Use cases

1) First 6 files of Software Foundations
   a) To answer the question: do we have enough primitives to build tactics?
   b) Basic tactics: intros, destruct, intro patterns, apply, simpl, unfold, assert, generalize.
   c) Imported tactics from Ltac: inversion, induction, rewrite.

2) Several important tactics of Iris
   a) To answer the question: how can we juice out types for tactics?

Some challenges we faced

1) What is a good representation for **goals**?
2) What are **tactics** and **tacticals**?
3) How to use types and have **typed tactics**?
4) How to avoid issues with **universes**?
5) How to obtain good **perfomance**?

How to represent goals: two approaches

1. **Opaque type:**
   a. Interaction with the goal through primitives of the language.
   b. Complex primitives for goal inspection and instantiation.
   c. Hard to get their context right!
   d. Most (or only?) representation found in other tactic languages.

2. **First-class representation:** ← Mtac2
   a. Direct interaction with the goal.
   b. No primitives: everything is coded in the meta-language!

What is a goal? (very partial answer)

- A goal is a *meta-variable*, but in Coq we just say is a term of some type:

```
Inductive goal :=
| Goal : forall {A}, A -> goal.
```

The proof. Invariant:
A *meta-variable*

What we need to prove

What is a goal? (very partial answer)

- A goal is a *meta-variable*, but in Coq we just say is a term of some type:

```
Inductive goal :=
| Goal : forall {A}, A -> goal.
```

- However, different subgoals may have different contexts:

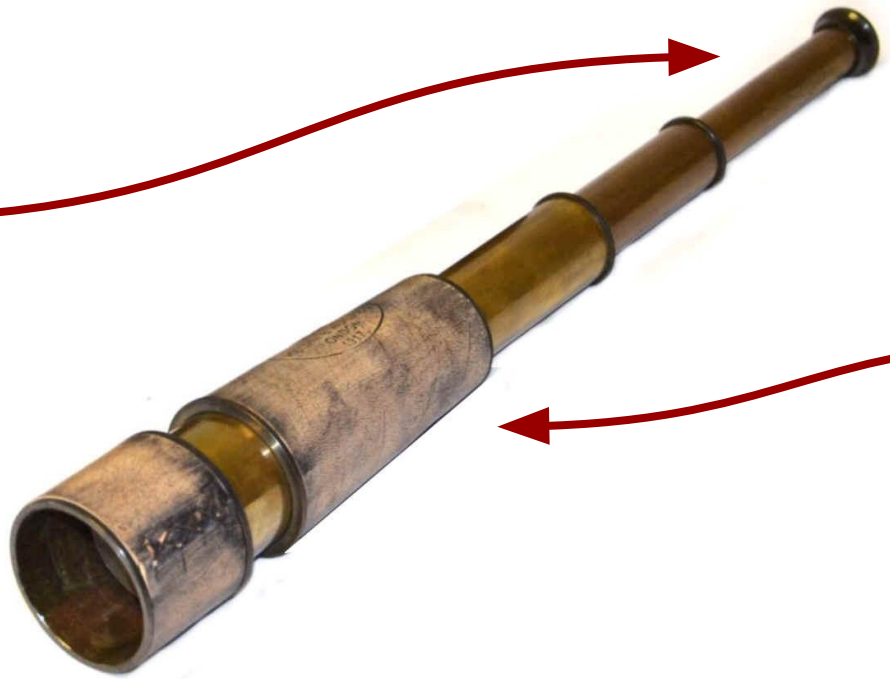
```
Theorem tl_length_pred : forall l: list nat,
  (length l) - 1 = length (tail l).
MProof.
  destructn 0 &> [m: idtac | intros n l ] &> reflexivity.
Qed.
```

2nd subgoal contains *n* and *l* in its context
We need to reflect that in the goal type!

Problem

How to compose tactics so that each work on the goal's specific context

Solution: make goals carry their own context!



The Goal

A Hypothesis

What is a goal? (partial yet sufficient answer)

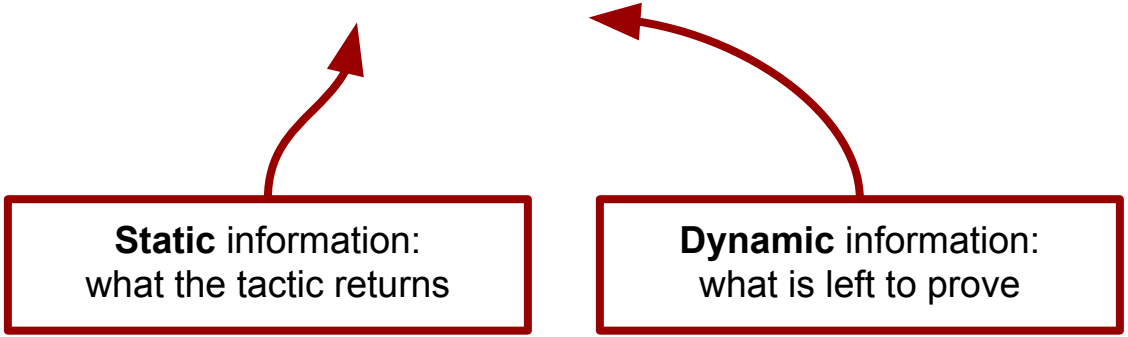
```
Inductive goal :=
| Goal : forall {A}, A -> goal
| AHyp : forall {A}, (A -> goal) -> goal.

Theorem tl_length_pred : forall l: list nat,
  (length l) - 1 = length (tail l).
MProof.
  destructn 0 &> [m: idtac | intros n l ] &> reflexivity.
Qed.
```

2nd subgoal is
AHyp (fun n=> AHyp (fun l => Goal *?y*))]

What is a typed tactic?

- A combination of *static* and *dynamic* information:

Definition ttac A := **M** (A * list goal).



Performance

- Coq is optimized for CBV.
- Mtac2 needs to be CBN.
  - When passing a goal as argument, we don't want the goal to be reduced!
- Done lots of progress, but not quite there yet.
  - In theory, we should be much faster than Ltac!

Conclusions

- Types in tactics allow us to build maintainable tactics.
- Mtac2 provides a simple and integrated model for typed tactics.
- Tested in a real dev: Iris.
- Three challenges: composition, performance and universes.
- Infinite possibilities for extensions.

M